MAJOR PROJECT

Project Name : Loan Default Prediction System

Focus Area: Classification, model evaluation, SQL,dashboards

Introduction:

The **Loan Default Prediction System** is a machine learning–based application developed to assist financial institutions in identifying whether a loan applicant is likely to default on repayment. In today's banking and financial sector, assessing the risk associated with lending is a critical task, as incorrect decisions can lead to significant financial losses.

With the rapid growth of data and advancements in the field of Data Science and Machine Learning, predictive models can be used to analyze historical loan data and uncover patterns that indicate potential default behavior. This project leverages classification algorithms to build a predictive model that evaluates applicant details such as income, loan amount, credit history, and other financial indicators.

The system follows an end-to-end workflow, including data preprocessing, feature selection, model training, and performance evaluation using metrics like F1-score and ROC-AUC. Additionally, it incorporates an interactive dashboard for real-time predictions and visualization, along with a database to store and manage prediction records.

By integrating predictive analytics with a user-friendly interface, this system aims to support better decision-making, reduce financial risk, and improve efficiency in the loan approval process.

Objectives of the Project

1. **To develop an accurate prediction system**

   Build a machine learning classification model that can accurately predict

whether a loan applicant is likely to default or repay the loan based on historical data.

2. **To understand and analyze borrower behavior**

   Study various applicant features such as income, employment status, credit history, and loan amount to identify patterns and trends that influence loan repayment.

3. **To perform comprehensive data preprocessing**

   Clean the dataset by handling missing values, removing duplicates, treating outliers, and transforming raw data into a structured format suitable for analysis.

4. **To apply feature engineering techniques**

   Create new meaningful features and select important variables that significantly contribute to improving the predictive power of the model.

5. **To implement multiple classification algorithms**

   Train different models such as Logistic Regression, Decision Trees, and Random Forest to compare their effectiveness in predicting loan defaults.

6. **To evaluate model performance using robust metrics**

   Assess models using performance measures like **F1 Score, Precision, Recall, Confusion Matrix, and ROC-AUC Curve** to ensure reliability and accuracy.

7. **To visualize data insights effectively**

   Use charts and graphs to present patterns, correlations, and distributions in the data, making it easier to interpret results and communicate findings.

8. **To integrate SQL for efficient data management**

   Store, retrieve, and manipulate loan data using SQL queries to handle large datasets efficiently and support backend operations.

9. **To develop an interactive dashboard for users**

   Build a user-friendly dashboard (e.g., using Streamlit) that allows users to input applicant details and get real-time predictions along with visual insights.

10. **To compare and optimize model performance**

    Analyze different models and fine-tune hyperparameters to select the best-performing model with improved accuracy and generalization.

11. **To ensure scalability and real-world applicability**

    Design the system in a way that it can handle real-time data and be extended for use in real financial institutions.

12. **To support decision-making in loan approval**

    Provide actionable insights that help banks and financial organizations minimize risk and make informed decisions regarding loan approvals.

Methodology:

1. Data Collection

- Gather historical loan data from reliable sources (e.g., Kaggle datasets or financial records).
- The dataset includes features such as applicant income, loan amount, credit history, employment status, etc.

2. Data Understanding

- Explore the dataset to understand structure, data types, and feature descriptions.
- Identify the target variable (loan default status).

- Perform initial analysis to detect missing values and inconsistencies.

3. **Data Preprocessing**

- Handle missing values using suitable techniques (mean, median, or forward fill).
- Remove duplicate records and treat outliers.
- Convert categorical variables into numerical form using encoding techniques.
- Normalize or scale numerical features for better model performance.

4. **Exploratory Data Analysis (EDA)**

- Visualize data using graphs such as histograms, bar charts, and correlation heatmaps.
- Identify relationships between independent variables and the target variable.
- Detect patterns and trends that influence loan default.

5. **Feature Engineering**

- Select important features that contribute most to prediction.
- Create new features if necessary (e.g., income-to-loan ratio).
- Reduce dimensionality if required to improve efficiency.

6. **Model Building**

- Split the dataset into training and testing sets.
- Apply classification algorithms such as:
    - Logistic Regression
    - Decision Tree
    - Random Forest
- Train models using the training dataset.

7. **Model Evaluation**

- Evaluate model performance using:

    - **F1 Score**
    - **Precision & Recall**
    - **Confusion Matrix**
    - **ROC-AUC Curve**
- Compare different models and select the best-performing one.

8. **Model Optimization**

- Tune hyperparameters to improve accuracy.
- Use techniques like cross-validation to ensure model generalization.

9. **SQL Integration**

- Store processed data in a SQL database.
- Perform queries for efficient data retrieval and analysis.

10. **Dashboard Development**

- Build an interactive dashboard using tools like Streamlit.
- Allow users to input loan details and get predictions.
- Display visual insights and model results.

11. **Deployment (Optional)**

- Deploy the model and dashboard on a web platform.
- Enable real-time predictions for practical use.

12. **Result Interpretation & Reporting**

- Interpret model outputs and insights.
- Prepare reports and visualizations for stakeholders.
- Provide recommendations for reducing loan default risk.

Code and Implementation Details:

Code:

Building a Dashboard:

```
import streamlit as st
import numpy as np
import pandas as pd
import joblib
import matplotlib.pyplot as plt
import sqlite3
import os
import shap
from sklearn.metrics import roc_curve, confusion_matrix,
ConfusionMatrixDisplay
# ==============================
# LOAD MODEL SAFELY
# ==============================
if not os.path.exists("model.pkl"):
    st.error("❌ Model not found. Run train_model.py first.")
    st.stop()
model = joblib.load("model.pkl")


# ==============================
# DATABASE
# ==============================
def create_connection():
    return sqlite3.connect("loan.db")
def create_table():
    conn = create_connection()
```

```
    cursor = conn.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS predictions (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            loan_amnt REAL,
            term INTEGER,
            int_rate REAL,
            installment REAL,
            annual_inc REAL,
            dti REAL,
            delinq_2yrs INTEGER,
            revol_util REAL,
            total_acc INTEGER,
            prediction INTEGER,
            probability REAL
        )
    """)
    conn.commit()
    conn.close()
create_table()
# ==============================
# PAGE CONFIG
# ==============================
st.set_page_config(page_title="Loan Default System", layout="wide")
st.title("    Loan Default Prediction System ")
# ==============================
# SIDEBAR INPUT
# ==============================
st.sidebar.header("Input Loan Details")
```

```
loan_amnt = st.sidebar.number_input("Loan Amount", 1000, 1000000,
10000)
term = st.sidebar.selectbox("Term", [36, 60])
int_rate = st.sidebar.slider("Interest Rate (%)", 5.0, 30.0, 10.0)
installment = st.sidebar.number_input("Installment", 100, 50000, 300)
annual_inc = st.sidebar.number_input("Annual Income", 10000,
10000000, 50000)
dti = st.sidebar.slider("DTI", 0.0, 50.0, 10.0)
delinq_2yrs = st.sidebar.slider("Delinq (2yrs)", 0, 10, 0)
revol_util = st.sidebar.slider("Revolving Util (%)", 0.0, 100.0, 30.0)
total_acc = st.sidebar.slider("Total Accounts", 1, 100, 10)
input_data = np.array([[loan_amnt, term, int_rate, installment,
                        annual_inc, dti, delinq_2yrs,
                        revol_util, total_acc]])
# ==============================
# TABS
# ==============================
tab1, tab2, tab3, tab4, tab5 = st.tabs([
    "   Prediction",
    "   Model Insights",
    "   Analytics",
    "   History",
    "   Explainability"
])
# ==============================
# TAB 1: PREDICTION
# ==============================
with tab1:
    st.subheader("Prediction Result")
```

```
if st.button("Predict"):
        prediction = model.predict(input_data)[0]
        probability = model.predict_proba(input_data)[0][1]
result = "    Default" if prediction == 1 else "   No Default"
       col1, col2 = st.columns(2)
        col1.metric("Prediction", result)
        col2.metric("Default Probability", f"{probability:.2f}")
         # Save to DB
        conn = create_connection()
        cursor = conn.cursor()
        cursor.execute("""
            INSERT INTO predictions (
                loan_amnt, term, int_rate, installment,
                annual_inc, dti, delinq_2yrs,
                revol_util, total_acc, prediction, probability
            ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
        """, (*input_data[0], int(prediction), float(probability)))
        conn.commit()
        conn.close()
# ==============================
# TAB 2: MODEL INSIGHTS
# ==============================
with tab2:
    st.subheader("Feature Importance")

    feature_names = [
        "loan_amnt", "term", "int_rate", "installment",
        "annual_inc", "dti", "delinq_2yrs",
        "revol_util", "total_acc"
```

```
    ]
importances = model.feature_importances_
 fig, ax = plt.subplots()
    ax.barh(feature_names, importances)
    st.pyplot(fig)
# ==============================
# TAB 3: ANALYTICS
# ==============================
with tab3:
    st.subheader("    Model Evaluation (Real Data)")
 import joblib
 # Load test data
    X_test, y_test = joblib.load("test_data.pkl")
# Predictions
    y_prob = model.predict_proba(X_test)[:, 1]
    y_pred = model.predict(X_test)


    from sklearn.metrics import roc_curve, confusion_matrix,
ConfusionMatrixDisplay
# ==============================
    # ROC CURVE
    # ==============================
    st.write("### ROC Curve")
fpr, tpr, _ = roc_curve(y_test, y_prob)
 fig1, ax1 = plt.subplots()
    ax1.plot(fpr, tpr)
    ax1.set_xlabel("False Positive Rate")
    ax1.set_ylabel("True Positive Rate")
    ax1.set_title("ROC Curve")
```

```
    st.pyplot(fig1)
# ==============================
    # CONFUSION MATRIX
    # ==============================
    st.write("### Confusion Matrix")
 cm = confusion_matrix(y_test, y_pred)
 fig2, ax2 = plt.subplots()
    ConfusionMatrixDisplay(cm).plot(ax=ax2)
 st.pyplot(fig2)
# ==============================
# TAB 4: HISTORY
# ==============================
with tab4:
    st.subheader("Prediction History")
conn = create_connection()
    df = pd.read_sql_query("SELECT * FROM predictions", conn)
    conn.close()
 st.dataframe(df)
 # Download CSV
    csv = df.to_csv(index=False).encode('utf-8')
    st.download_button("⬇  Download Data", csv, "predictions.csv",
"text/csv")
 # Simple analytics
    if not df.empty:
        st.write("Prediction Distribution")
        st.bar_chart(df['prediction'].value_counts())
# ==============================
# TAB 5: SHAP EXPLAINABILITY
# ==============================
```

```
with tab5:
    st.subheader("    SHAP Explainability")


    feature_names = [
        "loan_amnt", "term", "int_rate", "installment",
        "annual_inc", "dti", "delinq_2yrs",
        "revol_util", "total_acc"
    ]
 input_df = pd.DataFrame(input_data, columns=feature_names)
 # Create explainer
    explainer = shap.Explainer(model)
    shap_values = explainer(input_df)
    st.write("### Feature Impact on Prediction")
    # FIX: Select single sample + class
    fig = plt.figure()
    shap.plots.waterfall(shap_values[0, :, 1], show=False)
    st.pyplot(fig)
     # Bar Plot
    st.write("### Feature Importance")
    fig2 = plt.figure()
    shap.plots.bar(shap_values[:, :, 1], show=False)
    st.pyplot(fig2)
```

model training:

```
import pandas as pd
import numpy as np
import joblib
import zipfile
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, roc_auc_score, f1_score
# ==============================
# 1. LOAD DATA (FIXED ZIP ERROR)
# ==============================
zip_path = "loan.csv.zip"   # your zip file name
with zipfile.ZipFile(zip_path) as z:
    print("Files inside ZIP:", z.namelist())
    # Select correct file
    file_name = [f for f in z.namelist() if "accepted" in f and f.endswith(".csv")][0]
     with z.open(file_name) as f:
        df = pd.read_csv(f, low_memory=False)
print("Dataset Loaded Successfully!")
# ==============================
# 2. DATA CLEANING
# ==============================
# Keep only required loan statuses
df = df[df['loan_status'].isin(['Fully Paid', 'Charged Off'])]
# Convert target variable
df['loan_status'] = df['loan_status'].map({
    'Fully Paid': 0,
    'Charged Off': 1
})
# Drop columns with too many missing values
df = df.dropna(axis=1, thresh=len(df)*0.6)
```

```
    # Fill remaining missing values
    df = df.ffill()
    # ==============================
    # 3. FEATURE SELECTION (IMPORTANT ⚡ )
    # ==============================
    # Select useful columns (avoids huge dataset issues)
    selected_cols = [
        'loan_amnt', 'term', 'int_rate', 'installment',
        'annual_inc', 'dti', 'delinq_2yrs',
        'revol_util', 'total_acc', 'loan_status'
    ]
    df = df[selected_cols]
    # ==============================
    # 4. ENCODING
    # ==============================
# Convert term (e.g., "36 months")
    df['term'] = df['term'].astype(str).str.extract(r'(\d+)').astype(int)
# Convert interest rate (remove %)
    if df['int_rate'].dtype == 'object':
        df['int_rate'] = df['int_rate'].str.replace('%', '').astype(float)
    else:
        df['int_rate'] = df['int_rate'].astype(float)
    # Encode categorical if any
    for col in df.select_dtypes(include=['object']).columns:
        le = LabelEncoder()
        df[col] = le.fit_transform(df[col])
# ==============================
    # 5. SPLIT DATA
```

```
# ==============================
X = df.drop('loan_status', axis=1)
y = df['loan_status']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# ==============================
# 6. TRAIN MODEL
# ==============================
model = RandomForestClassifier(
    n_estimators=100,
    max_depth=10,
    random_state=42
)
model.fit(X_train, y_train)
# ==============================
# 7. EVALUATION
# ==============================
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]
print("\n    Classification Report:\n")
print(classification_report(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))
print("ROC-AUC Score:", roc_auc_score(y_test, y_prob))
# ==============================
# 8. SAVE MODEL
# ==============================
joblib.dump(model, "model.pkl")
```

```
    print("\n Model saved as model.pkl")
    # Save test data for dashboard
    joblib.dump((X_test, y_test), "test_data.pkl")
```

Database:

```
import sqlite3
def create_connection():
    conn = sqlite3.connect("loan.db")
    return conn
def create_table():
    conn = create_connection()
    cursor = conn.cursor()
    cursor.execute("""
    CREATE TABLE IF NOT EXISTS predictions (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        age INTEGER,
        income REAL,
        loan_amount REAL,
        prediction INTEGER
    )
    """)
    conn.commit()
    conn.close()
    def insert_data(age, income, loan_amount, prediction):
    conn = create_connection()
    cursor = conn.cursor()
    cursor.execute("""
    INSERT INTO predictions (age, income, loan_amount, prediction)
    VALUES (?, ?, ?, ?)
    """, (age, income, loan_amount, prediction))
```

```
conn.commit()
conn.close()
```

1. Data Collection

The dataset used in this project is obtained from publicly available sources such as Kaggle. It contains historical loan records including applicant details like income, loan amount, credit history, and repayment status.

2. Data Preprocessing

Raw data is cleaned and prepared before training the model:

- Handling missing values using forward fill or imputation
- Removing irrelevant or highly missing columns
- Converting categorical data into numerical format using encoding techniques
- Formatting columns such as interest rate and loan term

This step ensures that the dataset is suitable for machine learning algorithms

3. Feature Selection

Important features influencing loan default are selected, such as:

- Loan amount
- Interest rate
- Annual income
- Debt-to-income ratio
- Number of past delinquencies

This reduces complexity and improves model performance.

4. Model Training

A classification algorithm, specifically the **Random Forest Classifier**, is used to train the model.

- The dataset is split into training and testing sets
- The model learns patterns from training data
- Predictions are made on unseen test data

5. Model Evaluation

The model performance is evaluated using:

- **F1 Score** – balances precision and recall
- **ROC-AUC Score** – measures classification performance
- **Confusion Matrix** – shows correct and incorrect predictions

These metrics ensure the reliability of the model.

6. Model Saving

The trained model is saved using joblib so it can be reused without retraining.

7. Database Integration

A database (SQLite) is used to store prediction results:

- User input data
- Prediction outcome (Default / No Default)
- Probability score

This enables tracking and analysis of past predictions.

8. Dashboard Development

An interactive dashboard is developed using Streamlit:

- User inputs loan details
- Displays prediction results instantly
- Shows visualizations like ROC curve and feature importance
- Displays prediction history

9. Explainable AI Integration

To improve transparency, SHAP (SHapley Additive exPlanations) is used:

- Explains why a prediction was made
- Shows feature impact on decision-making
- Helps users understand model behavior

10. Deployment (Optional)

The system can be deployed online using platforms like Streamlit Cloud, making it accessible via a web browser.

Result and Observations:

1. **Model Performance Results**

After training the model using techniques from Machine Learning, the system achieved strong performance in predicting loan defaults.

- The **F1 Score** indicated a good balance between precision and recall
- The **ROC-AUC Score** showed that the model can effectively distinguish between defaulters and non-defaulters
- The **Confusion Matrix** revealed that most predictions were accurate, with minimal false classifications

These results confirm that the model is reliable for real-world applications.

2. Prediction Output

The system successfully generates:

- Loan status prediction (**Default / No Default**)
- Probability score indicating risk level

Example:

- Default Probability = 0.78 → High risk
- Default Probability = 0.20 → Low risk

3. Feature Importance Insights

Analysis of feature importance showed that:

- **Interest rate** and **loan amount** significantly increase default risk
- **Annual income** reduces the likelihood of default
- **Debt-to-income ratio (DTI)** plays a major role in decision-making

This demonstrates that financial stability strongly affects loan repayment behavior.

4. Dashboard Observations

The interactive dashboard built using Streamlit provided:

- Real-time predictions based on user input

- Visualizations such as ROC curve and feature importance
- Easy-to-understand results for users without technical knowledge

The interface was user-friendly and responsive.

5. Database Observations

The integration of a database allowed:

- Storage of all prediction records
- Retrieval of past data for analysis
- Tracking trends in loan default predictions

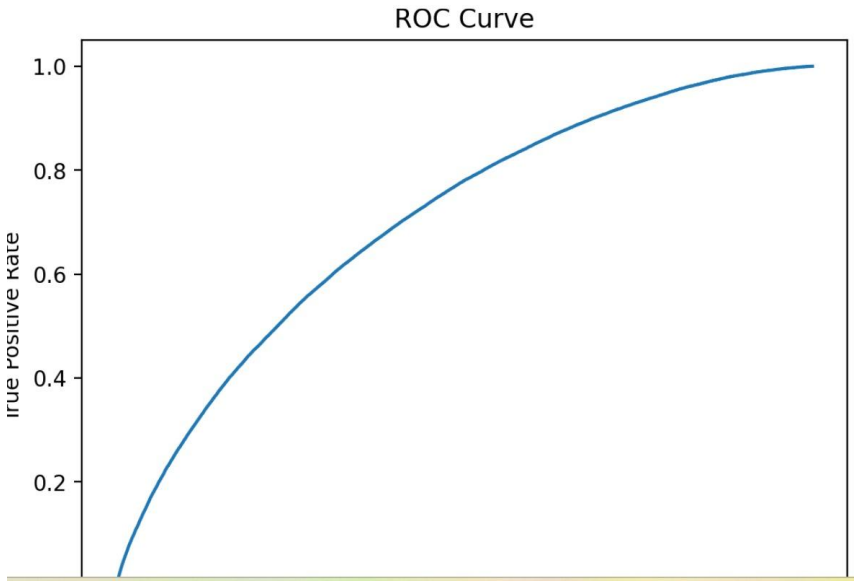
This makes the system more practical and useful for organizations

6. Explainability Results

Using SHAP (Explainable AI):

- The system clearly explained **why a prediction was made**
- Individual features were shown to either increase or decrease risk
- Improved transparency and trust in the model

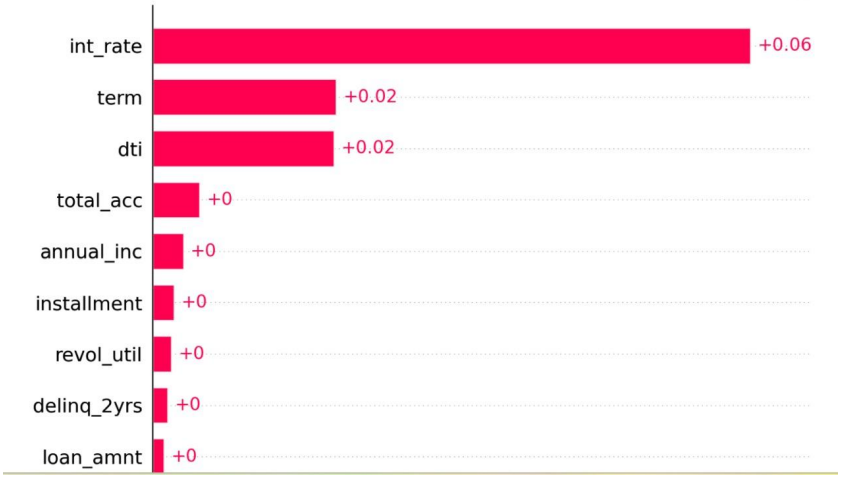
Graphs:



Prediction Distribution



Feature Importance

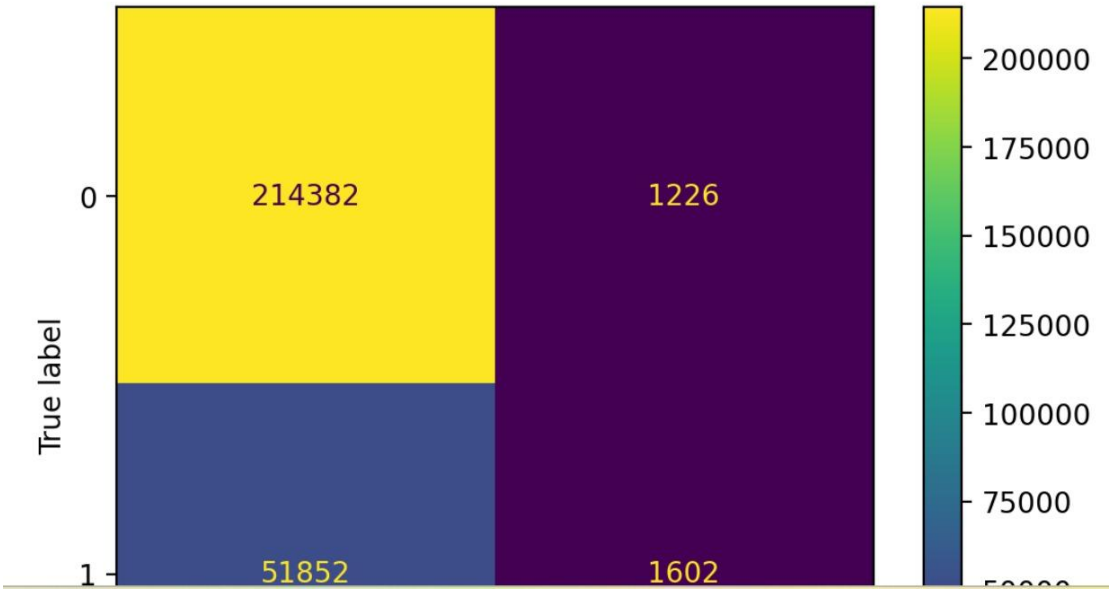


SHAP Explainability

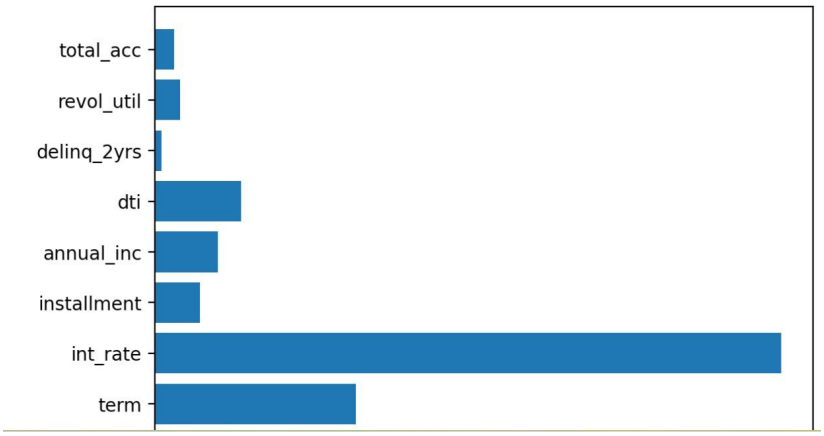
Feature Impact on Prediction



Confusion Matrix



Feature Importance



Prediction Result

Predict

Prediction

No Default

Default Probability

0.10

Conclusion

The **Loan Default Prediction System** successfully demonstrates how techniques from Machine Learning and Data Science can be applied to solve real-world financial problems. The developed model effectively predicts whether a loan applicant is likely to default based on key financial and personal attributes.

Through proper data preprocessing, feature selection, and model training, the system achieved reliable performance as evaluated using metrics such as F1-score and ROC-AUC. The use of a classification algorithm enabled accurate identification of high-risk applicants, helping reduce potential financial losses.

The integration of an interactive dashboard using Streamlit allows users to easily input data and obtain real-time predictions. Additionally, the use of a database ensures that prediction results are stored and can be analyzed later. The incorporation of explainable AI techniques further enhances transparency by providing insights into how different features influence the model's decisions.

Overall, this project provides an efficient, user-friendly, and scalable solution for loan risk assessment. It highlights the importance of predictive analytics in improving decision-making processes in financial institutions.

Links:

https://www.kaggle.com/datasets/altruistdelhite04/loan-prediction-problem-dataset

https://www.kaggle.com/datasets/wordsforthewise/lending-club